



Capstone Project – Python OOP and System Design

Practical Skills Module 4 Capstone Project

[Visit our website](#)

Capstone Overview

This capstone project forms part of your **Practical Skills Module 4** and is designed to assess your ability to develop intermediate Python programs that use object-oriented programming, algorithms, file handling, exception handling, and database operations.

In this capstone, you will complete a set of practical tasks that integrate the key topics covered in this module. You will create Python programs that model real-world systems, solve structured problems using algorithms, handle files and exceptions, and interact with a database using SQL.

Each task is based on a practical scenario and requires you to apply multiple concepts together. You will be expected to write clear, functional Python programs and produce outputs that show your understanding of how intermediate Python applications are designed and managed.

By the end of this capstone, you will have completed a set of Python files and related evidence that demonstrate your ability to apply intermediate Python programming principles in a structured and practical way.

You will show that you can:

- Define classes, methods, and objects in Python
- Apply inheritance, polymorphism, encapsulation, and abstraction
- Solve problems using algorithms and basic data structures
- Create, read, write, append, copy, rename, and compress files
- Handle exceptions so that programs fail gracefully
- Connect to and interact with a database using SQL operations

This capstone focuses on your ability to write structured Python programs, apply appropriate intermediate concepts, and produce clear evidence of your work.

Practical Module Scope

This capstone integrates the following practical skills:

- **PM-04-PS01:** Define classes, functions, or methods
- **PM-04-PS02:** Use inheritance, polymorphism, encapsulation, and abstraction
- **PM-04-PS03:** Solve problems using algorithms
- **PM-04-PS04:** Handle files

- **PM-04-PS05:** Handle exceptions
- **PM-04-PS06:** Handle a database in Python

Capstone Learning Outcomes

By completing this capstone, you will be able to:

- Define classes and methods in Python
- Create and use objects
- Apply inheritance in class design
- Implement polymorphism, encapsulation, and abstraction
- Use algorithms to solve practical problems
- Apply searching, sorting, mathematical, and data structure techniques
- Create and manage text, binary, and compressed files
- Check whether files and directories exist
- Catch and handle exceptions such as file and data errors
- Connect to a database and perform SQL operations such as insert, select, update, delete, commit, rollback, and close
- Produce clear and readable outputs that demonstrate correct program behavior

Instructions

You are required to complete all parts of this capstone project.

Follow these instructions carefully:

- Read each scenario before you begin
- Create the required Python files and save them using the exact filenames provided
- Ensure that your code is clearly structured, readable, and correctly indented
- Use meaningful variable, function, and class names
- Test each file to make sure it runs without errors
- Make sure all outputs are clearly labeled

- Include the required supporting evidence, such as the flowchart, SQL text file, and database screenshot
- Save all files in your task folder before submitting your work



Practical tasks

Practical Task 1: Employee and Library Management System

A company and a library both need simple Python systems to manage people and roles. The business wants to manage employee details, while the library wants to manage staff and members securely using object-oriented programming principles.

Create the following Python files:

- `employee_management_system.py`
- `library_management_system.py`

Part A: Employee Management System

Create a Python program that:

- Defines a class called `Employee`
- Includes a constructor with attributes such as employee name and employee ID
- Defines a method that displays employee details in a clear format
- Creates an object of the `Employee` class and calls the display method
- Creates subclasses such as `Manager` and `Developer` that inherit from `Employee`
- Adds one additional attribute to each subclass, such as department or programming language
- Overrides or extends the display method so that each subclass shows both shared and unique details

Part B: Library Management System

Create a Python program that:

- Defines a base class called Person
- Creates objects that represent people in the library system
- Creates subclasses such as Staff and Member
- Uses inheritance so that the subclasses reuse common attributes and behavior
- Uses polymorphism by giving subclasses their own version of a display method
- Uses encapsulation by making at least one attribute private and accessing it through getter and setter methods
- Uses abstraction with the abc module so that the base class includes at least one abstract method that must be implemented by subclasses

Use clear sample data, such as names and IDs, and display the results in a readable format.

Practical Task 2: Logistics Algorithm Planner

A logistics company needs a Python program to help organize deliveries, estimate costs, and manage package handling. The system should demonstrate how algorithms can be used to sort, search, calculate, and model basic delivery operations.

Create a Python file named `algorithms.py`.

Create a Python program that:

- Includes comments or pseudocode describing how a dynamic programming-style approach could be used to optimize a route
- Uses a simplified example with a small number of delivery locations
- Explains, through comments or printed output, how the route is selected
- Implements a sorting technique to arrange packages by priority
 - for example: High, Medium, Low
- Implements a searching technique to find a package by ID from a list
- Performs mathematical calculations such as:
 - $\text{delivery time} = \text{distance} / \text{speed}$

- $\text{delivery cost} = \text{distance} \times \text{fuel rate}$
- Uses a stack to add and remove packages
- Uses a queue to schedule deliveries in order

In addition, create a flowchart on paper or as an image that shows one selected process, such as package sorting, delivery scheduling, or cost calculation.

Your flowchart should include:

- Start
- Input
- Process
- Decision, where appropriate
- Output
- End

Practical Task 3: Customer Records and Order Processing

A retail business needs a Python program to manage customer records and process customer orders from files. The system must create and update files, back up important data, and handle errors without crashing.

Create the following files:

- `file_operations.py`
- `process_orders.py`
- `customer_orders.txt`

Part A: File Operations

Develop a Python program that performs the following file management tasks in sequence:

1. File Creation: Create a new file (e.g., `customers.txt`) and write initial data to it.
2. Basic Reading: Reopen the file and read its entire contents using `read()`.
3. Expansion: Create a second file (e.g., `inventory.txt`) and append new data to an existing file.
4. Line Processing: Read a file line-by-line to extract and process specific values.

5. File Verification: Check if a specific file exists on the system.
6. Maintenance: Rename an existing file and create a backup copy of a file.
7. Archiving: Create a ZIP archive containing your selected text files.
8. Binary Operations: Write data to a small binary file and successfully read it back.

Part B: Exception Handling

Develop a Python program to process `customer_orders.txt` using robust error handling. The program must:

1. File Input: Open `customer_orders.txt` for reading. (Use the sample data provided below).
2. Data Parsing: Read each line and split the contents into three variables: Order ID, Customer Name, and Amount.
3. Error Handling: Implement try-except blocks to manage the following:
 - `FileNotFoundError`: Catch this if the source file is missing.
 - `ValueError`: Catch this if the "Amount" field cannot be converted to a number.
4. Resilience: Ensure the program continues processing the remaining lines even if a specific line contains invalid data.
5. Cleanup: Use a finally block to ensure the file is closed properly if it was opened.
6. Confirmation: Print a final message confirming that the process has finished.

Sample Content (`customer_orders.txt`)

ORD001, John Doe, 100.50

ORD002, Jane Smith, 200.75

ORD003, Alice Johnson, abc

Practical Task 4: Student Database Operations

A training institution needs to manage student records stored in a database. The system must allow records to be added, viewed, updated, deleted, and confirmed through transaction control.

Create the following evidence:

- SQL code for the database operations

- A text file named Student.txt
- A screenshot saved as Database

You may use an online SQL editor such as **DB Fiddle** to test your SQL code before saving it.

Create SQL code that:

- Creates a table called Student
- Sets STU_NUM as the primary key
- Inserts the required student records into the table
- Selects all records where COURSE_CODE is 305
- Updates the COURSE_CODE to 304 for student 07
- Deletes the record for Jamie Lannister using all specified conditions
- Updates PROJ_NUM to 14 for students who meet the required date and course conditions
- Performs an update that is rolled back so the original data remains unchanged
- Performs a second update that is committed so the change is saved
- Selects the relevant records to confirm that rollback and commit worked correctly
- Drops the Student table at the end

Save your final SQL script in **Student.txt** and include a screenshot showing the steps you followed.



Take note

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.
